



**Escuela Técnica Superior de Ingeniería
Universidad de Huelva**

Máster en Ingeniería Informática

Trabajo Fin de Máster

*DeepSexist: Detección y Categorización de Mensajes
Sexistas en Vídeos Cortos Mediante Deep Learning*

Adrián Moreno Monterde
septiembre, 2026

A ChatGPT
Por estar siempre a mi lado mientras hacía este TFG/TFM

Resumen

Este apartado es muy importante, ya que es lo primero que se lee. Su redacción debe ser muy correcta y reflejar, de forma sintética, todo el trabajo realizado.

En una página como máximo, el resumen explica de modo conciso lo que intenta resolver el trabajo (¿Qué?), la metodología para abordar su solución (¿Cómo?), así como los principales resultados y conclusiones del trabajo.

Palabras clave: listado de palabras clave, separadas por coma, que representan el trabajo

Abstract

This section is very important, since it is the first thing that is read. Its wording must be very correct and reflect, in a synthetic way, all the work done.

In a maximum of one page, the abstract explains in a concise way what the work tries to solve (What?), the methodology to approach its solution (How?), as well as the main results and conclusions of the work.

Keywords: list of keywords, separated by comma, representing the work

Agradecimientos

Aunque no es un apartado obligatorio, dedicar unas líneas a expresar agradecimiento es una buena forma de reconocer el apoyo recibido a lo largo del proceso. Este espacio permite dar las gracias a todas aquellas personas que, de una u otra manera, han contribuido a que este trabajo haya podido desarrollarse y finalizar con éxito. Es habitual mencionar aquí a tutores, profesores, familiares, compañeros o amistades que hayan acompañado durante el camino.

Los agradecimientos pueden tener un tono personal e incluir alguna anécdota si se desea, pero se recomienda que no excedan una página de extensión.

Nombre del autor

Huelva, 2025

Índice general

Resumen	II
Abstract	III
Agradecimientos	IV
Índice de Figuras	VII
Índice de Tablas	VIII
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Competencias	3
1.4. Estructura de la Memoria	4
2. Marco Teórico	6
2.1. Aprendizaje Automático	6
2.2. Clasificación de textos, imágenes y vídeos	7
2.3. Aprendizaje Profundo	8
2.3.1. Epochs y tamaño de lote	10
2.3.2. Tasa de aprendizaje y <i>weight decay</i>	11
2.3.3. Función de pérdida	12
2.3.4. Overfitting y técnicas de regularización	12
2.3.5. Early stopping y validación	13
2.4. Procesamiento del Lenguaje Natural	14
2.4.1. Preprocesamiento del texto	14
2.4.2. Representación del texto	14
2.4.3. Clasificación de texto en PLN	15
2.5. Transformers	15
2.5.1. Modelos de lenguaje basados en Transformers	15
2.5.2. Transformers para visión por computador	15
2.5.3. Ventajas de los Transformers en tareas multimodales	16
2.6. Modelos Generativos de Lenguaje	16
2.6.1. Zero-shot, Few-shot y Fine-tuning	16
2.6.2. Aplicación a tareas de clasificación	17
2.6.3. Ventajas y limitaciones	17
2.7. Aprendizaje por Transferencia	18
2.7.1. Fine-tuning y optimización de hiperparámetros	18
2.8. Ensemble de Modelos	18
2.9. Aprendizaje con Desacuerdo (Learning with Disagreement)	18
2.10. Medidas de Evaluación	19
2.10.1. Precisión, Exhaustividad y F1-Score	19
2.10.2. Evaluación en clasificación multietiqueta	19
2.11. Tecnologías y Recursos Utilizados	20

3. Metodología, Experimentación y Resultados	22
3.1. Participación en una tarea (si es el caso)	22
3.2. Descripción General de la Metodología	23
3.3. Preparación de los Datos	24
3.3.1. Descripción de los Datos	25
3.3.2. Balanceo de Datos	25
3.3.3. Limpieza y Normalización	26
3.3.4. Tokenización y Codificación	26
3.3.5. División de Datos	26
3.4. Selección de los Modelos	27
3.5. Ajuste de Hiperparámetros	27
3.5.1. Métodos de Búsqueda	27
3.5.2. Hiperparámetros Evaluados	28
3.5.3. Tamaño del Conjunto de Datos Utilizado	28
3.5.4. Criterios de Selección	28
3.6. Entrenamiento de los Modelos	28
3.6.1. Baselines	29
3.6.2. Configuración de Entrenamiento	29
3.6.3. Monitorización del Entrenamiento	30
3.7. Evaluación y Análisis de Resultados	30
3.8. Análisis de Errores	33
3.9. Resultados Oficiales de la Tarea (opcional)	35
4. Conclusiones y Trabajo Futuro	37
4.1. Conclusiones	37
4.2. Trabajo Futuro	38
4.3. Planificación Temporal del Trabajo Realizado	39
Referencias	41
A. Anexos	43
A.1. Aplicación Web para la Visualización y Prueba del Modelo	43

Índice de Figuras

2.1. Flujo de procesamiento multimodal de vídeos (texto + imagen + vídeos) . .	8
2.2. Principal diferencia entre <i>Machine</i> y <i>Deep Learning</i>	9
2.3. Tecnologías y recursos utilizados en el desarrollo del trabajo	21
3.1. Pipeline general del marco de experimentación.	24
3.2. Evolución del entrenamiento monitorizada con TensorBoard.	30
3.3. Matriz de confusión para el modelo RoBERTa.	32
3.4. Matriz de confusión para el modelo RoBERTa.	32
3.5. Curvas ROC de los modelos evaluados.	33

Índice de Tablas

3.1. Resultados de evaluación sobre el conjunto de test.	31
3.2. Resultados por clase para los modelos evaluados (conjunto de test).	31
3.3. Resultados oficiales publicados por la organización de la tarea.	36
4.1. Planificación temporal del trabajo realizado.	39

Introducción

Todo proyecto de investigación necesita un contexto claro que justifique su desarrollo. Esta memoria recoge el trabajo realizado durante nuestra participación en la competición internacional *EXIST 2025*, cuyo reto principal es la detección y categorización automática del sexismo en redes sociales a partir del análisis combinado de texto, imágenes y vídeos.

A lo largo de este capítulo introductorio vamos a desglosar los motivos que nos han llevado a elegir este tema, destacando la necesidad urgente de aplicar la Inteligencia Artificial para frenar la discriminación y el odio en las redes sociales. Además, definiremos los objetivos concretos que han marcado el rumbo técnico del proyecto y repasaremos las competencias académicas adquiridas como frutos de la investigación. Como cierre, se presenta un breve itinerario al lector para guiarlo a través de los capítulos posteriores: el marco teórico, la metodología aplicada, la fase de experimentación y las conclusiones finales.

1.1. Motivación

El uso masivo de las plataformas sociales ha democratizado la comunicación, pero al mismo tiempo ha facilitado la propagación a gran escala del discurso de odio, debido al anonimato y, en la mayoría de los casos, impunidad. En este contexto digital, el sexismo destaca como un problema estructural que afecta de manera desproporcionada a las mujeres [1]. En redes basadas en texto como *Twitter* (actualmente *X*), la misoginia se hace patente mediante el acoso y la creación de narrativas discriminatorias, mientras que en plataformas de consumo rápido como *TikTok*, las dinámicas de los algoritmos pueden amplificar ideologías machistas y fomentar la hipersexualización. Paralelamente, el uso de formatos multimodales, como los memes, permite camuflar los prejuicios bajo la apariencia de humor o ironía, convirtiéndolos en mecanismos de difusión muy escurridizos.

Uno de los grandes retos para frenar este fenómeno es que el sexismo no siempre se manifiesta de forma explícita o mediante el uso de palabras malsonantes. Frecuentemente, se apoya en estereotipos profundamente arraigados, sesgos implícitos y microagresiones que normalizan la desigualdad. Comprender y detectar estas formas sutiles es crucial para abordar el problema, ya que su impunidad influye negativamente sobre la percepción social y perpetúa la discriminación. Sin embargo, la revisión y moderación manual de este tipo de contenido resulta en la actualidad completamente inviable, debido al inmenso volumen

de publicaciones que se generan constantemente, lo cual supera con creces la capacidad humana [2].

Por todo ello, resulta indispensable investigar y desarrollar herramientas automatizadas capaces de entender estas dinámicas tan complejas. Mediante este trabajo, proponemos aplicar y evaluar diversas técnicas de *Deep Learning* para construir modelos mediante un abordaje multimodal que ayuden a clasificar las publicaciones en las redes sociales. El propósito no es únicamente detectar la presencia de sexismo, sino caracterizarlo en función de la intención del mensaje y la categoría del abuso. Con la implementación de estas técnicas, buscamos proporcionar mecanismos escalables que apoyen la moderación de contenido, ayudando a mitigar la violencia de género en la red y contribuyendo a lograr plataformas digitales más inclusivas y libres de abusos.

1.2. Objetivos

El objetivo principal de este proyecto es diseñar, implementar y evaluar un sistema automatizado basado en técnicas de aprendizaje profundo (*Deep Learning*) para la detección, en la Tarea 3.1, y categorización del sexismo, en la Tarea 3.3, en vídeos de la plataforma *TikTok*. Para lograr este propósito, hemos planteado un enfoque multimodal capaz de procesar tanto de forma combinada como de manera independiente la información textual, a partir de las transcripciones de los vídeos, y la información visual y espaciotemporal, mediante la extracción y análisis de sus fotogramas (*frames*), enmarcando la investigación dentro de la competición internacional *EXIST 2025*.

Para alcanzar dicho objetivo general, se han definido los siguientes objetivos específicos:

- **Estudiar** el estado del arte y las técnicas actuales en el campo del *Deep Learning*, el Procesamiento del Lenguaje Natural (PLN) y la Visión Artificial aplicadas a la detección de contenido de odio y sexismo en formato vídeo.
- **Preprocesar y adecuar** el conjunto de datos proporcionado por la organización de EXIST 2025. Esto incluye el procesamiento de las transcripciones textuales y el diseño de un canal de extracción de fotogramas para posibilitar el análisis visual, gestionando además el paradigma *Learning with Disagreement* (LeWiDi).
- **Implementar y ajustar (*fine-tuning*)** modelos de estado del arte especializados en las diferentes modalidades trabajadas a partir del vídeo, contrastando arquitecturas de lenguaje con arquitecturas de visión estática y temporal.
- **Proponer y desarrollar** estrategias de fusión de modelos (*Ensemble*) para combinar las predicciones de las modalidades de texto, imagen y vídeo, con el fin de incrementar el rendimiento general y mitigar las deficiencias individuales de cada modelo.
- **Participar** en la tarea 3.1 de la competición *EXIST 2025*, estableciendo líneas base

de rendimiento mediante el uso de modelos de lenguaje supervisados (*RoBERTa*) y enfoques *zero-shot* mediante Grandes Modelos de Lenguaje (*LLaMA*).

- **Desarrollar**, en una fase de experimentación posterior a la competición, un marco de evaluación exhaustivo para las tareas 3.1 y 3.3, optimizando y fusionando (*Ensemble*) los mejores modelos de cada modalidad (texto, imagen y vídeo) para maximizar el rendimiento general.
- **Explorar** el impacto del perspectivismo y la subjetividad en la detección del sexismo, evaluando cómo el género de los anotadores influye en el sesgo del entrenamiento de los modelos.

1.3. Competencias

La realización de este Trabajo de Fin de Máster ha permitido poner en práctica y consolidar una serie de competencias fundamentales adquiridas durante la titulación, adaptándolas a un problema de investigación real y de alto impacto social.

De acuerdo con lo establecido en el documento de propuesta (Anexo II), la **competencia principal** que rige este trabajo es:

- **Capacidad de aplicar métodos matemáticos, estadísticos y de inteligencia artificial para modelar, diseñar y desarrollar aplicaciones, servicios, sistemas inteligentes y sistemas basados en el conocimiento.** Esta competencia se ha alcanzado de manera integral mediante el diseño y desarrollo de un sistema inteligente multimodal. Para ello, se han aplicado métodos avanzados de inteligencia artificial (*Deep Learning*) y estrategias estadísticas (como la agregación de distribuciones de probabilidad en etiquetas *soft* y el diseño de *ensembles*) con el fin de modelar y detectar el comportamiento sexista en la plataforma *TikTok*.

Además de esta competencia principal, el carácter de investigación de este proyecto nos ha permitido alcanzar las siguientes competencias adicionales vinculadas al plan de estudios:

- **Capacidad para el análisis, procesamiento y extracción automática de información a partir de datos no estructurados y multimodales.** Se ha logrado mediante la construcción de un flujo de trabajo técnico capaz de extraer fotogramas (*frames*) secuenciales a partir de los archivos de vídeo, así como trabajar con las transcripciones textuales de estos. Esto ha implicado limpiar el ruido implícito del lenguaje de las redes sociales y estructurar información bajo el complejo paradigma de las anotaciones múltiples (*Learning with Disagreement*).
- **Capacidad para conocer, diseñar y entrenar modelos *Deep Learning* aplicados a Visión Artificial y Procesamiento del Lenguaje Natural.** Esta capa-

cidad se ha consolidado a través de la implementación, adaptación y optimización de diversas arquitecturas expuestas en el capítulo del estado del arte. Entre ellas destacan modelos de lenguaje, como los *Transformers* y *Large Language Model (LLMs)*, y redes avanzadas de visión estática espaciotemporal, como los *Vision Transformers (ViT)* y *Video Vision Transformers (ViViT)*.

- **Capacidad para evaluar sistemas complejos y adquirir conocimientos a partir de grandes volúmenes de datos.** Esta competencia se ha desarrollado al enfrentar problemas de modelo del mundo real, como el desbalanceo de clases y colapso modal (*Mode Collapse*). La validación del sistema se ha fundamentado en la interpretación de métricas de evaluación avanzadas, como el *Information Contrast Measure (ICM)* o el *F1-score* para determinar los mejores rendimientos de los modelos y, por ende, poder establecer las mejores estrategias de fusión de modelos.
- **Conciencia ética y capacidad para analizar el impacto social y los sesgos en los sistemas de Inteligencia Artificial.** Se ha adquirido de manera transversal al explorar el perspectivismo y la subjetividad en la detección del sexismo. El análisis de algunos de los factores de los anotadores (como el género) pueden influir en el sesgo de entrenamiento de los modelos, permitiéndonos comprender la importancia crítica de diseñar sistemas de expertos más justos, transparentes y representativos.

1.4. Estructura de la Memoria

Los siguientes capítulos de la memoria de estructuran de la siguiente manera:

- En el [Capítulo 2](#), Marco Teórico, se exponen los conceptos fundamentales que sustentan este trabajo. Esto incluye los principios del *Deep Learning* y el Procesamiento del Lenguaje Natural, así como las arquitecturas de los modelos de lenguaje y de los modelos de clasificación de imagen y vídeo. También se describen técnicas clave como el aprendizaje por transferencia, estrategias de combinación de modelos, el paradigma del aprendizaje con desacuerdo y las métricas de evaluación empleadas.
- En el [Capítulo 3](#), Metodología, Experimentación y Resultados, se describe la metodología propuesta para alcanzar los objetivos planteados. Se detalla la participación en la competición *EXIST 2025* asociada a este trabajo, así como la preparación de los datos y las técnicas implementadas durante el ajuste de hiperparámetros y el entrenamiento de los modelos. Finalmente, se muestran y analizan los resultados obtenidos, incluyendo un análisis de errores y la exploración del perspectivismo.
- En el [Capítulo 4](#), Conclusiones y Trabajo Futuro, se presentan las conclusiones finales extraídas tras la realización de este proyecto, evaluando el grado de cumplimiento de los objetivos. Del mismo modo, se plantean las posibles directrices para investigaciones futuras y se detalla la planificación temporal seguida durante el desarrollo

del trabajo.

Marco Teórico

En este capítulo se describen los conceptos teóricos y el estado del arte necesarios para comprender el desarrollo y las decisiones de arquitectura tomadas para la realización de este Trabajo de Fin de Máster.

Dada la naturaleza compleja y multimodal del problema abordado (la detección y categorización del sexismo en redes sociales), el marco teórico abarca desde los principios fundamentales del Aprendizaje Automático y Profundo (*Deep Learning*) hasta los enfoques más modernos en Procesamiento del Lenguaje Natural y Visión Artificial. A lo largo de las siguientes secciones, se realiza una revisión crítica de las arquitecturas de estado del arte, prestando especial atención a los modelos basados en Transformers y a los Grandes Modelos de Lenguaje (*LLMs*). Asimismo, se fundamentan de manera teórica las estrategias empleadas para la experimentación, como el aprendizaje por transferencia, la fusión de modelos (*ensembles*) y el novedoso paradigma del aprendizaje con desacuerdo (*LeWiDi*), justificando su idoneidad para modelar la subjetividad y el sesgo en las plataformas digitales.

2.1. Aprendizaje Automático

El aprendizaje automático, comúnmente conocido como *Machine Learning (ML)*, es una de las disciplinas fundamentales dentro de la Inteligencia Artificial. Su objetivo principal es el desarrollo de algoritmos y modelos computacionales capaces de identificar patrones complejos y adquirir conocimiento empírico a partir de los datos, permitiendo a los sistemas mejorar su rendimiento en una tarea específica sin necesidad de ser explícitamente programados mediante reglas deterministas [3].

De manera clásica, atendiendo a la naturaleza de los datos de entrenamiento y al tipo de retroalimentación que recibe el sistema, distinguimos entre tres tipos de aprendizaje [4]:

- Aprendizaje Supervisado: En este enfoque, el algoritmo se entrena utilizando un conjunto de datos etiquetado, es decir, cada dato de entrada está emparejado de antemano con su resultado correcto o salida esperada. El objetivo del modelo es aprender una función matemática subyacente que relacione de forma generalizada las entradas con las salidas. Una vez entrenado, el sistema es capaz de inferir y predecir resultados

con cierto grado de precisión frente a nuevos datos con los que no ha interactuado previamente [5].

- Aprendizaje No Supervisado: A diferencia del tipo de aprendizaje anterior, en este caso el modelo recibe un conjunto de datos que carece por completo de etiquetas o salidas conocidas. El propósito del algoritmo no es predecir un valor en concreto, sino explorar la información para descubrir estructuras, patrones o agrupaciones de manera autónoma.
- Aprendizaje por Refuerzo: En este escenario no se parte de un conjunto de datos estático, sino que un agente aprende a tomar secuencias de decisiones interactuando dinámicamente con un entorno. Mediante un proceso continuo de prueba y error, el modelo ajusta su comportamiento con el objetivo de maximizar una señal de recompensa a largo plazo [6].

Para el desarrollo del presente trabajo, el enfoque central se enmarca dentro del Aprendizaje Supervisado. Dado que el objetivo es la detección y categorización del sexismo en la plataforma *TikTok*, los modelos requieren ser entrenados con colecciones de datos previamente anotados que asocian cada publicación (ya sea el vídeo, la transcripción textual o sus fotogramas) con una clase específica. Sin embargo, dada la alta subjetividad inherente a la interpretación de los discursos de odio en redes sociales, el concepto tradicional de “etiqueta única” del aprendizaje supervisado clásico presenta limitaciones, lo cual motivará la adopción de enfoques más flexibles como el aprendizaje con desacuerdo, que se abordará en secciones posteriores.

2.2. Clasificación de textos, imágenes y vídeos

La clasificación automática es una de las tareas fundamentales del aprendizaje automático supervisado. Consiste en asignar una o varias categorías a un dato de entrada a partir de un conjunto de ejemplos que estaban previamente etiquetados. Atendiendo a la cantidad de categorías y a su exclusividad, los problemas de clasificación pueden dividirse en:

- Clasificación binaria: Cuando el sistema debe discriminar entre dos únicas clases (0 o 1).
- Clasificación multiclase: Cuando el sistema debe discriminar entre más de dos clases, siendo éstas discriminantes entre sí. Esto quiere decir que dado un dato de entrada, este puede pertenecer a una de las clases sobre las que se esté clasificando.
- Clasificación multietiqueta: Cuando el sistema debe discriminar entre más de dos clases, siendo éstas no discriminantes entre sí. Esto quiere decir que una misma entrada puede no pertenecer a ninguna clase, pertenecer a una exclusivamente o pertenecer a varias simultáneamente [7].

Tradicionalmente, los modelos de clasificación se han diseñado para procesar una única modalidad de datos. En la clasificación de texto, la entrada consiste en secuencias de caracteres o palabras, requiriendo técnicas de Procesamiento de Lenguaje Natural para extraer la semántica, la sintaxis y el contexto del mensaje [8]. En la clasificación de imágenes, los algoritmos analizan la distribución espacial de los píxeles para identificar formas, texturas y patrones visuales estáticos. Por su parte, la clasificación de vídeo introduce una dimensión adicional y crítica: el tiempo. El procesamiento de este tipo de datos requiere no solo comprender la información visual estática de cada fotograma (*frame*), sino también modelar el movimiento y la evolución espaciotemporal a lo largo de la secuencia [9].

No obstante, en entornos reales donde la información es abundante y compleja, el uso de una modalidad suele ser insuficiente. Esto ha impulsado el desarrollo de la clasificación multimodal, un paradigma que integra múltiples fuentes de información de distinta naturaleza (por ejemplo, combinando un texto con una secuencia de imágenes). Para lograrlo, estos sistemas emplean arquitecturas especializadas en cada dominio y, posteriormente, aplican estrategias de fusión que combinan las representaciones extraídas, permitiendo al modelo tomar una decisión conjunta mucho más robusta frente a la ambigüedad [10].

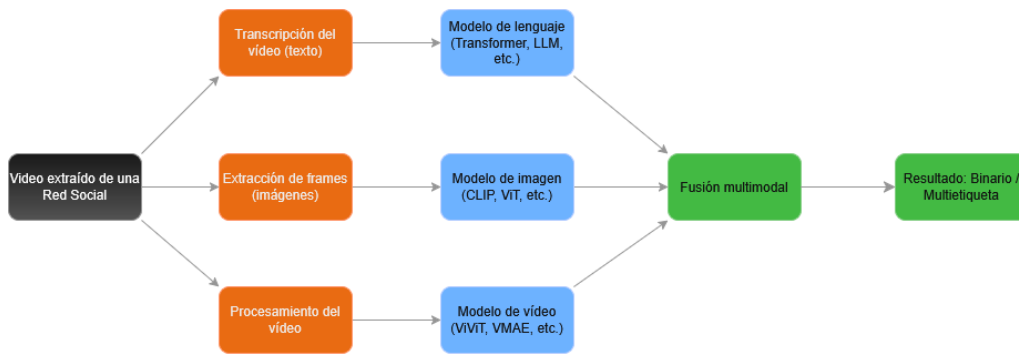


Figura 2.1: Flujo de procesamiento multimodal de vídeos (texto + imagen + vídeos)

Como se ilustra de manera conceptual en la Figura 2.1, un flujo de procesamiento multimodal estándar divide el contenido original en sus componentes básicos. Cada modalidad es procesada de forma independiente y en paralelo por modelos específicos para extraer sus características intrínsecas. Finalmente, estas representaciones convergen en un módulo de fusión, el cual combina la información para emitir la predicción final. Este esquema teórico sirve como pilar fundamental para el diseño de arquitecturas capaces de analizar el complejo contenido multimedia actual.

2.3. Aprendizaje Profundo

Dentro de la disciplina del aprendizaje automático, el aprendizaje profundo (*Deep Learning*) representa una evolución significativa [11]. Esta rama se fundamenta en el uso de

redes neuronales artificiales, arquitecturas computacionales inspiradas en cómo el cerebro humano estructura y procesa la información. Al organizar estas neuronas artificiales en múltiples capas sucesivas, el sistema adquiere la capacidad de procesar la información de forma jerárquica, logrando un nivel de abstracción cada vez mayor. A medida que aumenta la profundidad de la red (el número de capas), el algoritmo es capaz de modelar relaciones más complejas y extraer mayor cantidad de información a partir de los datos, incluso si no es perceptible por el ser humano [12].

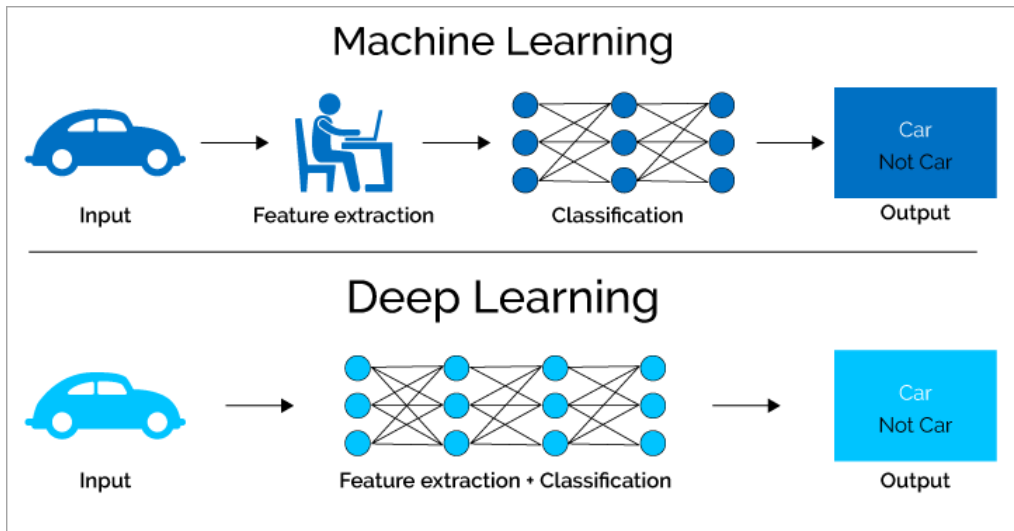


Figura 2.2: Principal diferencia entre *Machine* y *Deep Learning*

La principal diferencia entre el aprendizaje automático clásico y el aprendizaje profundo radica en el proceso de extracción de características (*feature extraction*). Como se ilustra en la Figura 2.2, en los algoritmos tradicionales de *Machine Learning*, es necesario que un experto humano o un proceso algorítmico previo diseñe y extraiga manualmente las características relevantes de los datos de entrada para que el clasificador pueda tomar decisiones. Por el contrario, los modelos de *Deep Learning* automatizan este proceso por completo, integrando la extracción de características de forma implícita dentro de la propia arquitectura de la red. Esta capacidad de aprendizaje de extremo a extremo (*end-to-end*) permite al modelo descubrir por sí mismo qué patrones son los más instructivos para resolver la tarea[13].

En el contexto de este Trabajo de Fin de Máster, el aprendizaje profundo constituye el núcleo técnico del sistema propuesto. Se han empleado modelos basados en arquitecturas profundas, tanto para el procesamiento de la información textual, con modelos *Transformers* y Grandes Modelos de Lenguaje, como para el análisis visual, con modelos *Vision Transformers*.

Dado el enorme tamaño de estas arquitecturas y la complejidad matemática que entraña su ajuste, resulta indispensable comprender cómo se lleva a cabo su fase de entrenamiento. El éxito de estos modelos no depende únicamente de su estructura interna, sino

de la correcta configuración de los factores que guían su aprendizaje. Por ello, en las siguientes subsecciones se detallan los hiperparámetros, mecanismos de control y conceptos fundamentales que controlan el proceso de entrenamiento.

2.3.1. Epochs y tamaño de lote

Durante el proceso de entrenamiento de una red neuronal profunda, el modelo ajusta sus parámetros internos de forma iterativa a medida que procesa la información. El ritmo y el volumen de datos que se procesan en cada actualización de los pesos están regidos por dos hiperparámetros fundamentales de la arquitectura: la época (*epoch*) y el tamaño de lote (*batch size*).

Una época (*epoch*) se define como un ciclo de aprendizaje completo en el que la totalidad del conjunto de datos de entrenamiento pasa a través de la red neuronal, tanto hacia adelante (*forward pass*) para generar las predicciones, como hacia atrás (*backward pass*) para propagar el error. Dado que los algoritmos de optimización basados en el gradiente son iterativos, una sola época rara vez es suficiente para que el modelo minimice el error adecuadamente. Por lo tanto, el entrenamiento exige realizar múltiples épocas. Sin embargo, un número excesivamente alto de épocas puede derivar en un problema de sobreajuste (*overfitting*), donde el modelo comienza a memorizar los datos de entrenamiento perdiendo su capacidad de generalización [14].

Por otro lado, la cantidad de información que componen los conjuntos de datos actuales (especialmente aquellos con imágenes y vídeos) hace que sea computacionalmente inviable, debido a las limitaciones de memoria de las tarjetas gráficas (*GPU*), procesar todos los ejemplos simultáneamente. Para solucionar esta limitación, el conjunto de entrenamiento se divide en fracciones más pequeñas denominadas lotes (*batches* o *minibatches*). El tamaño de lote determina el número de muestras que la red procesa antes de evaluar el error acumulado y actualizar sus pesos [15].

La elección del tamaño de lote tiene un impacto directo en la estabilidad y velocidad del entrenamiento. Un lote grande ofrece una estimación muy precisa del gradiente, pero requiere una enorme cantidad de memoria *VRAM*. Por el contrario, un tamaño de lote pequeño introduce cierto “ruido” matemático en cada actualización. Curiosamente, este ruido suele ser beneficioso, ya que actúa como un mecanismo de regularización que ayuda a la red a escapar de mínimos locales y a encontrar soluciones que generalizan mejor ante datos nuevos [16].

En el contexto del ajuste fino (*fine-tuning*) de arquitecturas masivas, como los *Transformers* o los *LLMs* empleados en este trabajo, el enorme peso de los modelos obligan a utilizar tamaños de lote muy reducidos. Asimismo, dado que estas redes ya cuentan con un amplio conocimiento adquirido durante su preentrenamiento, el número de épocas ne-

cesarias suele ser considerablemente bajo (generalmente entre 3 y 10). Un entrenamiento prolongado en estas condiciones no solo sería ineficiente, sino que podría provocar lo que se conoce como *catastrophic forgetting*, el cual destruiría las representaciones lingüísticas o visuales que el modelo ya había aprendido previamente.

2.3.2. Tasa de aprendizaje y *weight decay*

La tasa de aprendizaje (*learning rate*) es, posiblemente, el hiperparámetro más crítico en la configuración de cualquier algoritmo de optimización basado en el gradiente. Este valor determina el tamaño del paso que da el modelo en el espacio de parámetros durante cada iteración para minimizar la función de pérdida. Su correcta elección tiene un impacto significativo en la convergencia del modelo: si la tasa de aprendizaje es demasiado alta, los saltos en la actualización de los pesos serán desproporcionados, lo que puede provocar que el modelo sobrepase el mínimo óptimo y el entrenamiento se vuelva inestable. Por el contrario, si la tasa es excesivamente baja, la red tardará demasiado tiempo en converger o correrá el riesgo de quedarse estancada en un mínimo local [17].

Para complementar el proceso de optimización y mitigar el riesgo de sobreajuste, es práctica común incorporar una técnica de regularización conocida como *weight decay* (decaimiento de pesos). Esta técnica consiste en añadir un término de penalización a la función de pérdida que es proporcional al tamaño de los pesos de la red. De este modo, el algoritmo se ve forzado a mantener los valores de los parámetros lo más pequeños y distribuidos posible. Al evitar que unos pocos pesos adquieran valores desproporcionados, se asume que el modelo resultante es más simple y estable, lo que mejora drásticamente su capacidad para generalizar ante datos no vistos en lugar de memorizar el ruido del conjunto de entrenamiento [18].

En el ámbito del procesamiento del lenguaje natural y la visión por computador con arquitecturas profundas, ambos conceptos suelen trabajar de la mano mediante optimizadores avanzados como *AdamW*. Al realizar el *fine-tuning* de modelos masivos que ya poseen un conocimiento previo (*Transformers* o *LLMs*), la teoría dictamina el uso de tasas de aprendizaje muy reducidas, a menudo acompañadas de planificadores (*schedulers*) que disminuyen este valor dinámicamente con el tiempo. El uso simultáneo de un *learning rate* bajo y un *weight decay* adecuado garantiza que el modelo se adapte a la nueva tarea de clasificación sin destruir las representaciones que adquirió durante su fase de preentrenamiento [19].

2.3.3. Función de pérdida

Durante la fase de entrenamiento, el modelo necesita una métrica cuantitativa que evalúe la precisión de sus predicciones frente a las etiquetas reales del conjunto de datos. Esta métrica es la función de pérdida (*loss function*). Su objetivo matemático es calcular el error cometido en cada predicción, devolviendo un valor que el algoritmo de optimización (mediante *backpropagation*) intentará minimizar iterativamente actualizando los pesos de la red [20]. En los problemas de clasificación, las funciones de pérdida basadas en la teoría de la información, concretamente en el concepto de entropía, son el estándar.

Para tareas de clasificación binaria o multiclase, donde las categorías son mutuamente excluyentes (es decir, una entrada solo puede pertenecer a una clase), se emplea la **Entropía Cruzada** (*Cross-Entropy Loss*). Esta función mide la diferencia entre la distribución predicha por el modelo y la distribución real de las etiquetas. Matemáticamente, penaliza de forma logarítmica las predicciones erróneas que el modelo realiza con alta confianza, forzando al sistema no solo a acertar la clase correcta, sino a hacerlo con la mayor certeza posible [21].

Por otro lado, para abordar la tarea de clasificación multietiqueta (donde un mismo registro puede presentar varias categorías de sexismo simultáneamente), el problema se descompone en múltiples clasificaciones independientes. Al tratar cada etiqueta por separado, la entropía cruzada mencionada anteriormente no es aplicable. En su lugar, se utiliza la **Entropía Cruzada Binaria** (*Binary Cross-Entropy* o *BCE*). Esta variante evalúa la probabilidad de presencia o ausencia de cada etiqueta de manera independiente, permitiendo que el modelo determine si una categoría específica aplica o no a la entrada, sin que dicha decisión afecte a las probabilidades del resto de etiquetas [22].

2.3.4. Overfitting y técnicas de regularización

Uno de los desafíos más críticos durante el entrenamiento de redes neuronales profundas es el fenómeno conocido como sobreajuste (*overfitting*). Este problema ocurre cuando un modelo es excesivamente complejo en relación con la cantidad de datos de entrenamiento disponibles. En lugar de extraer patrones generales y subyacentes, la red comienza a memorizar el ruido, las anomalías y los detalles específicos del entrenamiento. Como resultado, el modelo alcanza un rendimiento excelente sobre los datos con los que ha sido entrenado, pero fracasa drásticamente al intentar generalizar y predecir sobre datos nuevos o no vistos previamente [23].

Para mitigar el problema y forzar al modelo a aprender representaciones más robustas, se emplean diversas técnicas de regularización. Una de las estrategias más extendidas, que actúa de manera complementaria a la penalización de pesos (*weight decay*) vista en apartados anteriores, es el *Dropout* (abandono).

El *Dropout* es una técnica de regularización estructural que consiste en desactivar aleatoriamente una proporción de las neuronas de la red, junto con sus respectivas conexiones, durante cada iteración (*forward* y *backward pass*) de la fase de entrenamiento. Al “apagar” distintas neuronas en cada paso, se evita que las unidades de la red desarrollen una dependencia mutua excesiva. Esto obliga al modelo a distribuir el aprendizaje, garantizando que la predicción final no dependa de un pequeño grupo de neuronas fuertemente conectadas, sino del conjunto de múltiples sub-redes independientes [24].

En las arquitecturas profundas basadas en mecanismos de atención, como los *Transformers* utilizados en este trabajo tanto para el procesamiento textual como visual, el *Dropout* es un componente fundamental que se encuentra integrado por defecto en el diseño de sus bloques internos. La combinación del *Dropout* arquitectónico con estrategias de optimización regularizadas asegura que el modelo mantenga su capacidad de generalización al adaptarse a la nueva tarea de clasificación.

2.3.5. Early stopping y validación

Para monitorizar de forma objetiva el rendimiento de un modelo durante su fase de entrenamiento y detectar a tiempo un posible sobreajuste, resulta imprescindible el uso de un conjunto de validación. Este subconjunto de datos, que se mantiene independiente de las muestras empleadas para actualizar los pesos de la red, permite evaluar iterativamente la capacidad de generalización del modelo. Este proceso se da al finalizar cada época o tras un número determinado de iteraciones y se calcula la función de pérdida y otras métricas de rendimiento sobre este conjunto de validación.

La observación continua de estas métricas es la base del *early stopping* (parada temprana), una de las técnicas de regularización dinámica más eficaces en el aprendizaje profundo. Esta estrategia consiste en interrumpir el entrenamiento de forma automática antes de completar todas las épocas definidas si se detecta que el rendimiento en validación se estanca o empieza a degradarse, mientras que el error de entrenamiento sigue disminuyendo. Este punto de inflexión es el indicador clave de que el modelo ha dejado de aprender patrones generales y ha empezado a memorizar el conjunto de datos de entrenamiento [25].

En la práctica, el *early stopping* se configura mediante un parámetro conocido como paciencia (*patience*), el cual define cuántas evaluaciones sin mejora se permiten antes de detener el proceso. Además de ahorrar valiosos recursos computacionales, esta técnica permite restaurar la configuración de pesos óptima (aquella que obtuvo el mejor resultado en la fase de validación) en lugar de retener los pesos deteriorados de la última época.

Todos estos elementos analizados en las subsecciones anteriores son fundamentales para diseñar modelos robustos y reproducibles. Además, permiten explicar por qué un modelo ha funcionado mejor que otro cuando se ajustan frente a un problema complejo, más

allá de lo que dictan las métricas de evaluación. Como señalan Bergstra y Bengio [26], el conocimiento de estos hiperparámetros y mecanismos de control es clave para mejorar la eficacia y garantizar el éxito de los modelos de aprendizaje automático.

2.4. Procesamiento del Lenguaje Natural

Sugerencia

Esta es una sección para explicar en qué consiste el PLN y por qué es relevante en el contexto del trabajo. Se pueden introducir los conceptos fundamentales que forman parte del preprocesamiento y análisis de texto.

El Procesamiento del Lenguaje Natural (PLN) es un área de la inteligencia artificial que se ocupa de la interacción entre computadoras y lenguaje humano. Su objetivo es permitir que las máquinas comprendan, interpreten, generen o analicen lenguaje en formato textual o hablado. En el contexto de este trabajo, el PLN se emplea para analizar el texto presente en memes y detectar patrones relacionados con sesgos de género o lenguaje sexista.

El PLN moderno ha evolucionado rápidamente gracias a los modelos de lenguaje basados en arquitecturas profundas, como Transformers. Estos modelos permiten representar el texto en espacios vectoriales de alta dimensión, conservando tanto información semántica como sintáctica [27].

2.4.1. Preprocesamiento del texto

[Aquí se puede explicar en qué consiste la limpieza del texto antes de pasarlo al modelo. Se puede mencionar la tokenización, la eliminación de ciertos caracteres, etc.]

2.4.2. Representación del texto

Tradicionalmente, el texto se representaba mediante bolsas de palabras o TF-IDF. Actualmente, se utilizan representaciones densas (embeddings) que capturan mejor el significado. Los modelos como Word2Vec, GloVe y, más recientemente, BERT y sus derivados, permiten obtener representaciones contextuales del texto. En este trabajo, se utilizan embeddings obtenidos mediante modelos como DistilBERT o RoBERTa para tareas de clasificación.

2.4.3. Clasificación de texto en PLN

La clasificación de texto es una tarea central en PLN. Puede ser binaria, multiclase o multitiqueta, dependiendo del número y tipo de categorías. En este trabajo, se utiliza para determinar si un meme expresa contenido sexista, y en caso afirmativo, identificar los tipos de sesgo presentes. Este problema se ha abordado en campañas como IberLEF o SemEval, donde se proponen datasets y tareas similares.

2.5. Transformers

Sugerencia

En esta sección se puede introducir la arquitectura Transformer, su motivación original y las ventajas que aporta frente a modelos anteriores. Se debe justificar su aplicación al trabajo que se está desarrollando (por ejemplo, clasificación de texto, procesamiento de imágenes o fusión multimodal).

La arquitectura Transformer fue propuesta por Vaswani et al. en 2017 bajo el lema “Attention is all you need” [28]. A diferencia de modelos recurrentes tradicionales (RNN, LSTM), los Transformers procesan la información en paralelo y capturan relaciones a largo plazo entre tokens mediante mecanismos de atención.

2.5.1. Modelos de lenguaje basados en Transformers

Los Transformers han dado lugar a una nueva generación de modelos de lenguaje preentrenados, como BERT [27], RoBERTa y muchos otros. Estos modelos se entrenan con grandes volúmenes de texto mediante tareas de aprendizaje no supervisado y luego se adaptan mediante fine-tuning a tareas específicas.

Sugerencia

En esta subsección debes explicar cuáles son modelos que has utilizado (por ejemplo, DistilBERT, RoBERTa, etc.)

2.5.2. Transformers para visión por computador

Aunque los Transformers fueron diseñados inicialmente para texto, han sido adaptados con éxito al dominio visual. Modelos como ViT (Vision Transformer) y CLIP combinan información de imagen y texto usando atención cruzada o espacios embebidos conjuntos.

CLIP, en particular, permite obtener representaciones multimodales alineadas que resultan útiles en tareas como clasificación de memes, búsqueda semántica y detección de contenido.

2.5.3. Ventajas de los Transformers en tareas multimodales

La capacidad de los Transformers para manejar secuencias de cualquier tipo y capturar dependencias entre elementos los convierte en una herramienta potente para tareas multimodales. En este trabajo se han utilizado arquitecturas que procesan texto e imagen en paralelo y combinan la información para realizar predicciones. Este enfoque permite capturar mejor las relaciones entre el texto de un meme y su componente visual.

2.6. Modelos Generativos de Lenguaje

Sugerencia

En esta sección podéis explicar qué son los LLMs generativos, en qué se diferencian de los modelos tradicionales de clasificación y cómo pueden aplicarse a tareas como la clasificación de textos o de imágenes. Se pueden dar ejemplos de *prompting* y explicar si se han usado en el trabajo o si se plantean como línea futura.

Los Modelos Generativos de Lenguaje (LLMs, por sus siglas en inglés) son una clase de modelos entrenados para predecir la siguiente palabra en una secuencia, permitiéndoles generar texto coherente y contextualizado. A diferencia de los modelos tradicionales (como BERT, que son discriminativos), los LLMs utilizan arquitecturas autoregresivas y se entrenan sobre grandes volúmenes de datos sin etiquetas explícitas.

Modelos como GPT-3, GPT-4, LLaMA o PaLM han demostrado capacidades sorprendentes en múltiples tareas de PLN, incluyendo clasificación, resumen, traducción o razonamiento. En particular, pueden realizar tareas de clasificación a través de técnicas como el **zero-shot prompting**, **few-shot prompting** o **instruction tuning**, lo que permite aprovechar su conocimiento sin necesidad de reentrenamiento.

2.6.1. Zero-shot, Few-shot y Fine-tuning

Sugerencia

Explica las tres formas principales de utilizar modelos generativos en tareas supervisadas. Puedes acompañarlo de ejemplos de prompts y aclarar en qué contexto del trabajo se ha usado cada enfoque (si procede).

Una de las ventajas más potentes de los LLMs generativos es su capacidad para realizar tareas sin necesidad de reentrenamiento. Existen tres modos principales de uso:

- **Zero-shot:** el modelo realiza la tarea directamente con un único prompt, sin ejemplos adicionales. Es útil cuando no se dispone de datos etiquetados o se busca rapidez. *Ejemplo:* ¿Este meme contiene contenido sexista? "Las mujeres no deberían opinar de fútbol". Responde: Sí o No.
- **Few-shot:** se incluyen algunos ejemplos dentro del prompt para que el modelo aprenda.^a partir de ellos durante la misma interacción. *Ejemplo:* Ejemplo 1: "Las mujeres no sirven para programar"→ Sexista
Ejemplo 2: "Todos merecen las mismas oportunidades"→ No sexista
Frase: "Las chicas no deberían jugar videojuegos"→
- **Fine-tuning:** consiste en ajustar los pesos del modelo entrenándolo con un conjunto de datos específico. Requiere más recursos, pero suele ofrecer mejores resultados. En este trabajo se ha considerado como opción para futuras mejoras del rendimiento.

Sugerencia

Si en el trabajo se ha probado alguno de estos enfoques, coméntalo aquí.

2.6.2. Aplicación a tareas de clasificación

En lugar de entrenar un clasificador específico, es posible plantear la tarea como una pregunta directa al modelo. Por ejemplo:

¿Este comentario contiene lenguaje sexista?
Texto: "Las mujeres deberían quedarse en casa".
Respuesta: Sí.

Este enfoque permite evaluar memes o textos con un modelo generativo sin necesidad de entrenamiento adicional. Sin embargo, plantea retos como la reproducibilidad, la sensibilidad al prompt, o la variabilidad en la salida.

2.6.3. Ventajas y limitaciones

- **Ventajas:** no requieren datasets específicos, pueden adaptarse rápidamente a nuevas tareas, y permiten aprovechar conocimiento general.
- **Limitaciones:** no siempre son consistentes, pueden ser sensibles al prompt, y su uso masivo implica un coste computacional elevado.

2.7. Aprendizaje por Transferencia

Sugerencia

En esta sección debes explicar qué es el aprendizaje por transferencia y por qué se ha vuelto un enfoque estándar en PLN, visión artificial y aprendizaje multimodal.

El aprendizaje por transferencia consiste en reutilizar el conocimiento adquirido por un modelo en una tarea previa (generalmente entrenado sobre grandes volúmenes de datos) para resolver una nueva tarea, normalmente con menos datos disponibles. En lugar de entrenar un modelo desde cero, se parte de uno preentrenado y se adapta a una nueva tarea específica mediante distintas técnicas, como fine-tuning o extracción de características.

Este enfoque ha sido fundamental para el desarrollo de aplicaciones prácticas en procesamiento del lenguaje natural y visión por computador, especialmente desde la aparición de modelos como BERT [27], RoBERTa, CLIP o GPT. En lugar de aprender representaciones desde cero, estos modelos aprenden representaciones generales del lenguaje o de la imagen que luego pueden refinarse con una menor cantidad de datos etiquetados [29].

2.7.1. Fine-tuning y optimización de hiperparámetros

Una de las estrategias más utilizadas en transferencia es el fine-tuning, que consiste en ajustar los pesos de un modelo preentrenado para que se adapten a una tarea nueva.

2.8. Ensemble de Modelos

Sugerencia

Si se ha utilizado en el trabajo, explicar aquí lo que es y cómo se ha utilizado.

2.9. Aprendizaje con Desacuerdo (Learning with Disagreement)

Sugerencia

Si se ha utilizado en el trabajo, debéis explicar lo que es: conceptos como hard y soft label, etc., incluyendo ejemplos de cómo puede ser aplicado en el trabajo.

2.10. Medidas de Evaluación

Sugerencia

En esta sección debes explicar las métricas que vas a utilizar para evaluar los modelos. Es importante que no sólo pongas su definición, sino que también justifiques su elección en función del tipo de tarea: clasificación binaria o multietiqueta, etc.

Para evaluar los modelos de clasificación desarrollados en este trabajo se utilizan métricas estándar del aprendizaje automático. Estas permiten cuantificar el rendimiento del sistema, especialmente en tareas sensibles como la detección de contenido sexista.

2.10.1. Precisión, Exhaustividad y F1-Score

En la clasificación binaria, se emplean métricas como la precisión (precision), la exhaustividad (recall) y la F1-score, que combinan los aciertos verdaderos positivos con los errores cometidos. Estas métricas son especialmente útiles cuando hay clases desbalanceadas.

Sugerencia

Puedes definir fórmulas matemáticas en LaTeX usando el entorno `equation`, que la numera automáticamente.

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.1)$$

Donde la precisión y la exhaustividad se calculan como:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

y donde TP, FP y FN representan los verdaderos positivos, falsos positivos y falsos negativos respectivamente.

2.10.2. Evaluación en clasificación multietiqueta

En el caso de la clasificación multietiqueta, donde cada instancia puede tener múltiples etiquetas a la vez, se utilizan variantes como el F1 micro y macro, y métricas específicas como el exact match ratio, que indica el porcentaje de ejemplos en los que se predicen exactamente las mismas etiquetas que en los datos reales.

2.11. Tecnologías y Recursos Utilizados

Sugerencia

En esta sección podéis enumerar las principales herramientas, bibliotecas y plataformas utilizadas en el desarrollo del trabajo. No se trata de hacer una lista sin más, sino de explicar brevemente para qué se ha utilizado cada tecnología y por qué fue elegida. Puedes incluir logos o iconos pequeños para que quede más visual.

El desarrollo de este proyecto se ha llevado a cabo utilizando un conjunto de herramientas ampliamente adoptadas en el ámbito de la inteligencia artificial y el procesamiento de lenguaje natural, como se muestra en la figura 2.3.

- **Python 3.10:** Lenguaje de programación principal del proyecto, utilizado para la implementación de scripts de procesamiento, entrenamiento de modelos y análisis de resultados.
- **Jupyter Notebook:** Entorno interactivo para el desarrollo exploratorio, la visualización de resultados y la ejecución modular del código.
- **PyTorch:** Framework de aprendizaje profundo utilizado para construir, entrenar y evaluar los modelos basados en Transformers y redes neuronales.
- **Hugging Face:** Biblioteca clave para cargar y ajustar modelos preentrenados como BERT, RoBERTa, DistilBERT y CLIP.
- **Google Colab:** Plataforma de ejecución en la nube que ha permitido el entrenamiento de modelos con acceso a GPUs de forma gratuita.
- **Optuna / Weights & Biases (W&B):** Herramientas de optimización y seguimiento de experimentos utilizadas para ajustar hiperparámetros y visualizar métricas durante el entrenamiento.
- **GPU del laboratorio I2C:** Parte del entrenamiento final de los modelos se realizó en entornos locales del grupo de investigación, usando recursos computacionales propios para mejorar la velocidad de entrenamiento.

Sugerencia

Puedes insertar una figura con los logos de estas herramientas organizados en una fila o en bloques, indicando debajo brevemente su rol. Queda muy bien visualmente en este tipo de memorias.



Figura 2.3: Tecnologías y recursos utilizados en el desarrollo del trabajo

Sugerencia

Se puede terminar el capítulo con un párrafo parecido a este:

En resumen, este capítulo presenta los conceptos aprendidos para entender los enfoques adoptados en el desarrollo de este trabajo. Este aprendizaje me ha servido para comprender las decisiones técnicas y metodológicas que se presentan en los capítulos siguientes.

Metodología, Experimentación y Resultados

Sugerencia

Como siempre, al comenzar un capítulo y antes de empezar con la primera sección, se debe escribir un párrafo introductorio para explicar lo que el lector se va a encontrar en este capítulo.

3.1. Participación en una tarea (si es el caso)

Si el trabajo se ha realizado en el contexto de una tarea como *IberLEF*, *CLEF*, *SemEval*, etc., esta sección debe comenzar con una descripción detallada de dicha tarea. Es fundamental que el lector entienda el marco en el que se sitúa el trabajo.

Sugerencia

IMPORTANTE: evitar usar las palabras “competición”, “concurso”, etc. Mejor usar términos como “reto”, “desafío”, etc.

Sugerencia

Buscar todos los detalles posibles y relevantes de la tarea en su propia página web y describirlos. Debe quedar muy claro en qué consiste la tarea, su utilidad, su campo de aplicación, el interés que tiene, el número de ediciones, quién la organiza, el workshop en el que está incluida, etc. No escatimar palabras.

Se puede introducir esta parte con un párrafo como el siguiente:

Este trabajo se ha desarrollado en el marco de la tarea [NombreTarea], organizada en el contexto del workshop [NombreWorkshop], celebrado en [Año]. La tarea tiene como objetivo principal...

Es recomendable incluir también información como:

- Número de equipos participantes.

- Descripción de los datos proporcionados.
- Métricas de evaluación empleadas.
- Categorías o sub-tareas existentes.

Si habéis conseguido una buena posición, es importante destacarlo con claridad:

Nuestro sistema obtuvo el segundo mejor resultado en la métrica F1 para la sub-tarea de clasificación multicategoría, situándose por encima de la media de los participantes.

Y si el trabajo ha dado lugar a un artículo científico, también se puede mencionar:

Como parte de la participación, se redactó un artículo científico titulado “Título del artículo”, que fue aceptado en el workshop [Nombre del workshop], y publicado en las actas del evento.

Sugerencia

La tarea es un medio para enmarcar el trabajo, pero no el objetivo final del TFG/TFM. El foco debe estar en el aprendizaje, la experimentación, la propuesta y evaluación de metodologías o enfoques propios, y en la comprensión profunda del problema que se aborda.

3.2. Descripción General de la Metodología

En esta sección se debe mostrar una visión global y estructurada de la metodología y el marco de experimentación seguido para el desarrollo del trabajo. El objetivo es que el lector comprenda los distintos pasos y fases por los que se ha pasado para construir y evaluar los modelos y, en consecuencia, para resolver la tarea.

Sugerencia

Enumeración y descripción breve de los pasos principales en el proceso de entrenamiento del modelo: preprocesamiento, selección o definición del modelo, ajuste de hiperparámetros, entrenamiento, evaluación, etc.

Una posible redacción introductoria podría ser:

El marco de experimentación se ha dividido en varias fases: preprocesamiento de los datos, diseño y entrenamiento de los modelos, y evaluación de su rendimiento. En esta sección se ofrece una visión general del proceso completo, que será detallado en las secciones posteriores.

Ejemplo de estructura de pasos

- Preprocesamiento de datos
- Selección o definición del modelo base
- Ajuste de hiperparámetros
- Entrenamiento del modelo
- Evaluación y análisis de resultados

Sugerencia

Incluir un gráfico o diagrama de flujo que visualice la secuencia completa de pasos y procesos seguidos. Este elemento servirá como referencia visual para complementar las descripciones textuales

La [Figura 3.1](#) muestra el pipeline del marco de experimentación seguido en este trabajo.

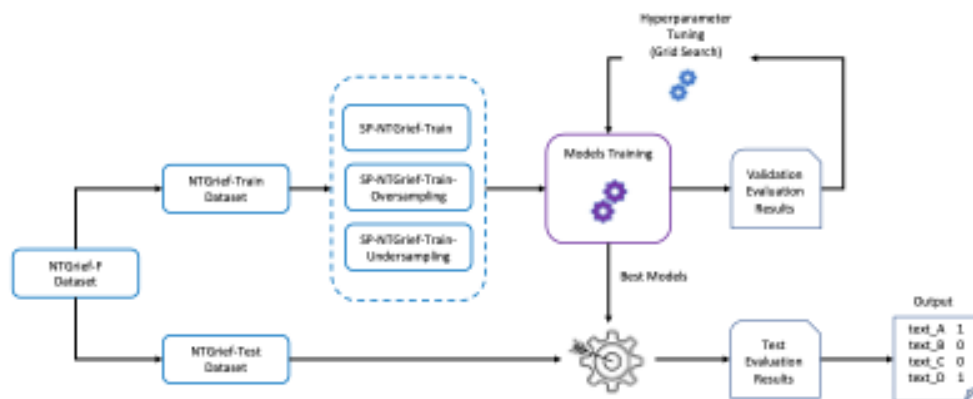


Figura 3.1: Pipeline general del marco de experimentación.

3.3. Preparación de los Datos

En esta sección se debe explicar detalladamente cómo se han preparado los datos antes de ser utilizados por los modelos. Esto incluye su exploración inicial, transformación, limpieza, codificación y división en subconjuntos.

3.3.1. Descripción de los Datos

Sugerencia

Describir los datos proporcionados por la tarea: formato, número de instancias, tipo de anotaciones, etc. Incluir ejemplos reales. Se recomienda acompañar con gráficas exploratorias (por ejemplo, palabras más frecuentes por clase, distribución de etiquetas, tamaños de texto, etc.).

El dataset proporcionado por la organización de la tarea está compuesto por 3500 memes anotados con una etiqueta de acuerdo o desacuerdo. Además, incluye su texto asociado. El dataset tiene la información de todos los anotadores que han participado en el proceso de anotación...

En la Figura "Tal" se muestran las palabras más frecuentes en los textos etiquetados como "sexista".

3.3.2. Balanceo de Datos

Sugerencia

Si el dataset original está desbalanceado, indicar qué técnicas se han aplicado para corregirlo (por ejemplo, backtranslation, sobre-muestreo, sub-muestreo...). Especificar qué modelos o herramientas se han usado y cómo ha quedado el dataset final tras el balanceo.

Dado que la clase "sexista" estaba poco representada (17% del total), se aplicó una estrategia de augmentación mediante backtranslation con el modelo Helsinki-NLP/opus-mt-en-es¹. Tras el proceso, el dataset aumentó a 5100 instancias.

Sugerencia

IMPORTANTE: poner notas al pie es muy fácil en Latex con el entorno footnote. Se utiliza mucho para hacer referencias a direcciones URL cuando no se ponen como referencias, como en el caso del párrafo anterior

¹<https://huggingface.co/Helsinki-NLP/opus-mt-en-es>

3.3.3. Limpieza y Normalización

Sugerencia

Describir los pasos de limpieza y normalización aplicados al texto, como la conversión a minúsculas, eliminación de signos de puntuación, tratamiento de emojis, corrección de caracteres erróneos, etc. También puede mencionarse el paso de json a csv o la transformación de estructuras.

Se eliminaron los caracteres especiales no alfabéticos, se convirtieron los textos a minúsculas y se estandarizaron emoticonos comunes. El conjunto original estaba en formato JSON y fue convertido a CSV para facilitar el procesamiento posterior.

3.3.4. Tokenización y Codificación

Sugerencia

Explicar cómo se han tokenizado los textos y cómo se convierten en índices o vectores de entrada. Se recomienda incluir un ejemplo sencillo mostrando los tokens generados y sus códigos (input_ids).

Para tokenizar los textos se utilizaron los tokenizadores correspondientes a cada modelo. Por ejemplo, el texto “I totally agree” se tokeniza como [“i”, “totally”, “agree”] y se codifica como [1045, totally_id, agree_id].

3.3.5. División de Datos

Sugerencia

Describir cómo se han dividido los datos en conjuntos de entrenamiento, validación y test. Indicar proporciones y criterios. Acompañar con gráficas de barras o de otro tipo que muestren visualmente las proporciones y el equilibrio de clases en cada subconjunto.

Los datos se dividieron en 80 % para entrenamiento, 20 % para validación y 20 % para test. Se garantizó el equilibrio de clases en cada subconjunto mediante muestreo estratificado.

3.4. Selección de los Modelos

En esta sección se debe justificar la elección de los modelos empleados para resolver la tarea. Es importante mencionar tanto los modelos que finalmente se han utilizado como aquellos que fueron considerados durante el análisis inicial.

Sugerencia

Listar los modelos de Transformers evaluados (como *BERT*, *RoBERTa*, *GPT*, *LLaMA*, *Mistral*, etc.) y explicar brevemente las razones por las que se seleccionaron o descartaron. Es obligatorio referenciar los modelos en la bibliografía o mediante notas a pie de página.

Durante la fase de estudio de la tarea se evaluaron varios modelos preentrenados de Transformers. Entre ellos, destacaron bert-base-uncased, roberta-base y mistral-7b-instruct.

La elección final del modelo roberta-base se basó en su buen rendimiento en tareas similares y en la disponibilidad de recursos computacionales adecuados.

También se consideró el uso de modelos de mayor tamaño, como LLaMA 2 13B, pero fueron descartados por las limitaciones de memoria en los entornos disponibles.

3.5. Ajuste de Hiperparámetros

En esta sección se debe detallar el proceso seguido para ajustar los hiperparámetros del modelo. Es importante justificar las decisiones tomadas y explicar los métodos utilizados para optimizar el rendimiento del sistema.

3.5.1. Métodos de Búsqueda

Sugerencia

Describir los métodos utilizados para explorar combinaciones de hiperparámetros: búsqueda exhaustiva (*grid search*), búsqueda aleatoria (*random search*), frameworks como *Optuna*, *Wandb*, etc.

Para el ajuste de hiperparámetros se utilizó una estrategia de búsqueda aleatoria empleando el framework Optuna, que permitió explorar eficientemente combinaciones relevantes sin necesidad de evaluar todas las posibles.

3.5.2. Hiperparámetros Evaluados

Sugerencia

Enumerar los hiperparámetros que se han ajustado y especificar el espacio de búsqueda de cada uno.

Se ajustaron los siguientes hiperparámetros: tasa de aprendizaje (`learning_rate`) en el rango $[1e-5, 5e-5]$, número de épocas entre 2 y 6, tamaño de batch entre 8 y 32, etc.

Sugerencia

En lugar de escribir los valores en un párrafo, es conveniente hacer una tabla con el espacio de búsqueda

3.5.3. Tamaño del Conjunto de Datos Utilizado

Sugerencia

Indicar si se ha utilizado el conjunto completo de entrenamiento o solo una partición para la búsqueda de hiperparámetros. Justificar la decisión.

Dado el coste computacional del ajuste, se utilizó el 30 % del conjunto de entrenamiento para la búsqueda de hiperparámetros. Se mantuvo una distribución estratificada para preservar el equilibrio entre clases.

3.5.4. Criterios de Selección

Sugerencia

Explicar qué métrica se ha utilizado para determinar la combinación de hiperparámetros óptima (por ejemplo, accuracy, F1-score, etc.), y en qué conjunto se evaluó.

La selección del mejor conjunto de hiperparámetros se basó en la F1-score macro obtenida en el conjunto de validación. Esta métrica fue priorizada al tratarse de un problema con clases desbalanceadas.

3.6. Entrenamiento de los Modelos

Esta sección debe detallar cómo habéis realizado el entrenamiento de los modelos seleccionados, incluyendo las configuraciones, baselines (si se han utilizado) y el seguimiento del

proceso de entrenamiento.

3.6.1. Baselines

Sugerencia

Si se ha utilizado un sistema baseline, describirlo brevemente: tipo de modelo, hiperparámetros empleados, resultados obtenidos. Es importante establecerlo como referencia para evaluar las mejoras del modelo final.

Como baseline se utilizó un clasificador ... Este sistema alcanzó una precisión del 68 % en el dataset de test. Sirvió como punto de comparación para evaluar la mejora introducida por los modelos desarrollados en este trabajo.

3.6.2. Configuración de Entrenamiento

Sugerencia

Detallar la configuración empleada: librerías utilizadas (por ejemplo, **Trainer** de HuggingFace), número de épocas, valores finales de hiperparámetros, optimizador, regularización (dropout), etc. Indicar también la métrica usada para guardar el mejor modelo.

*El entrenamiento se llevó a cabo con la clase **Trainer** de la librería **transformers** de HuggingFace. Se utilizó el optimizador **AdamW** con una tasa de aprendizaje de $2 \cdot 10^{-5}$, un tamaño de batch de 16 y dropout del 0.1. El entrenamiento se realizó durante 4 épocas, y se guardó el modelo con mejor F1-score macro sobre el conjunto de validación.*

Sugerencia

Si habéis hecho optimización de hiperparámetros, aquí no debéis repetir el punto "Ajuste de Hiperparámetros". Anteriormente habéis descrito la estrategia de búsqueda de hiperparámetros, y en esta sección debéis detallar otros aspectos, especialmente los mejores valores de hiperparámetros obtenidos para cada modelo, etc.

3.6.3. Monitorización del Entrenamiento

Sugerencia

Describir cómo se ha monitorizado el entrenamiento: uso de callbacks, logs, métricas registradas, visualización en herramientas como TensorBoard o la consola de HuggingFace. Incluir capturas o gráficos si es posible.

*Se empleó la integración de **Trainer** con **TensorBoard** para monitorizar en tiempo real la evolución de la pérdida y la métrica de evaluación. Además, se activó el callback **EarlyStoppingCallback** para detener el entrenamiento si no había mejora durante 3 épocas consecutivas.*

En la [Figura 3.2](#) se muestra un ejemplo de la evolución del F1-score durante el entrenamiento.

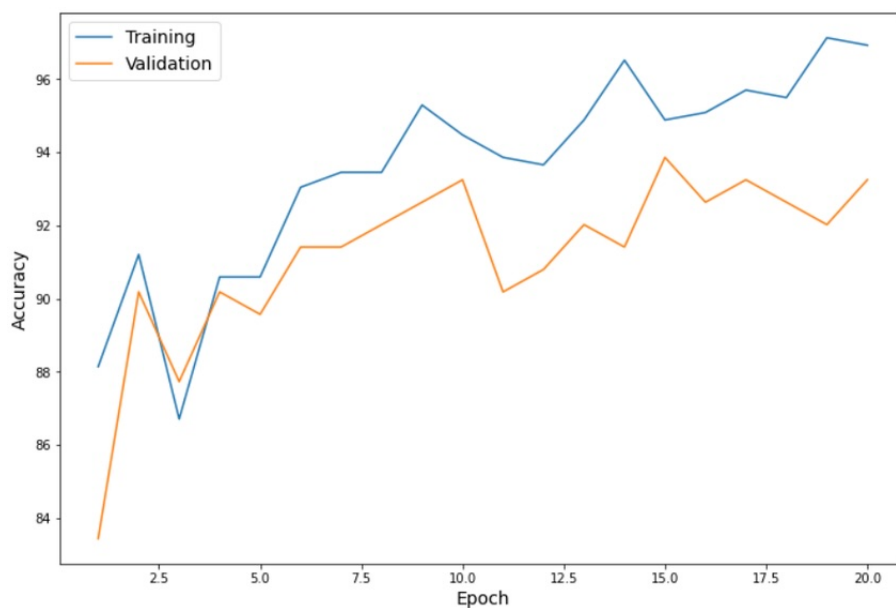


Figura 3.2: Evolución del entrenamiento monitorizada con TensorBoard.

3.7. Evaluación y Análisis de Resultados

En esta sección se presentan los resultados obtenidos tras evaluar los modelos entrenados sobre vuestro conjunto de test durante la fase de entrenamiento y mejora de los modelos. Se deben incluir todas las métricas relevantes, gráficos y análisis que permitan interpretar el rendimiento de las distintas estrategias y modelos.

Sugerencia

Incluye tablas de resultados para todos los modelos evaluados, siguiendo un estilo similar al de los artículos científicos. No olvides añadir títulos claros y destacar los mejores resultados en negrita.

Sugerencia

Presentar las métricas más relevantes: precisión (accuracy), precisión por clase, recall, F1-score, macro/micro promedio, etc. Utiliza `classification_report` o similares y organiza los resultados en tablas bien formateadas.

En la [Tabla 3.1](#) se presentan los resultados obtenidos por los modelos entrenados sobre el conjunto de test antes del proceso de limpieza. Se incluyen las métricas de F1-score, precisión y recall.

Modelo	Precision	Recall	F1-score
BERT-base	0.76	0.78	0.75
RoBERTa-base	0.81	0.83	0.80
Baseline	0.65	0.66	0.64

Tabla 3.1: Resultados de evaluación sobre el conjunto de test.

En la [Tabla 3.2](#) se presentan los resultados obtenidos por los modelos entrenados sobre el conjunto de test desglosados por clase.

Tabla 3.2: Resultados por clase para los modelos evaluados (conjunto de test).

Modelo	Clase	Precision	Recall	F1-score
BERT-base	Clase 1	0.81	0.79	0.80
	Clase 2	0.70	0.72	0.71
	Clase 3	0.68	0.66	0.67
RoBERTa-base	Clase 1	0.85	0.83	0.84
	Clase 2	0.76	0.77	0.76
	Clase 3	0.72	0.71	0.71

Matrices de Confusión

Sugerencia

Dibujar las matrices de confusión para los principales modelos, preferiblemente con anotaciones y colores. Explicar lo que se observa: equilibrio en los aciertos, etc.

En la [Figura 3.3](#) se muestra la matriz de confusión del modelo RoBERTa. Se observa que las clases minoritarias son las que peor han sido clasificadas, etc.

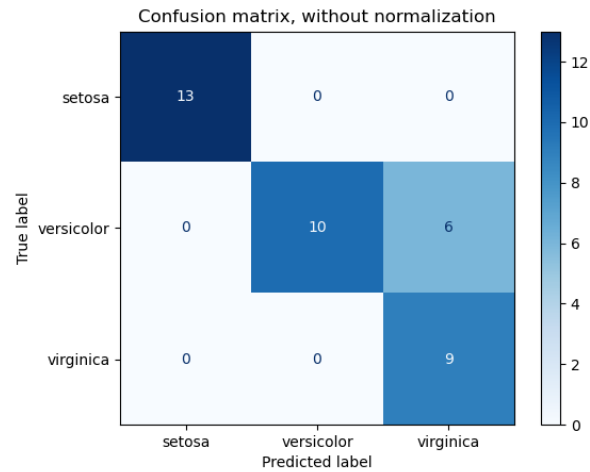


Figura 3.3: Matriz de confusión para el modelo RoBERTa.

En la [Figura 3.4](#) se muestra la matriz de confusión del modelo Llama con las 6 etiquetas. Se observa que se ha comportado muy bien con las clases 1 y 2. Sin embargo, la clase 4 ha obtenido muchos errores, etc.

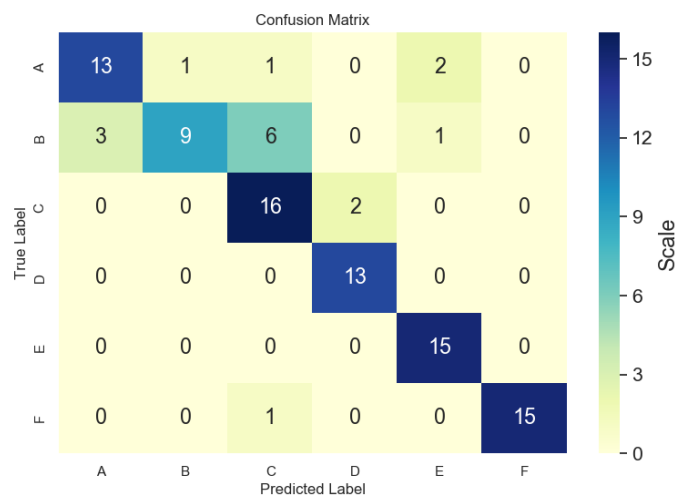


Figura 3.4: Matriz de confusión para el modelo RoBERTa.

Curvas ROC y AUC

Sugerencia

Incluir las curvas ROC para cada modelo. Si el problema lo permite (clasificación binaria), puede añadirse también la curva Precision-Recall. Explicar su interpretación y qué significan las áreas bajo las curvas (AUC).

La [Figura 3.5](#) muestra las curvas ROC de los modelos evaluados. El modelo RoBERTa alcanza el mayor valor de AUC, superando el 0.90.

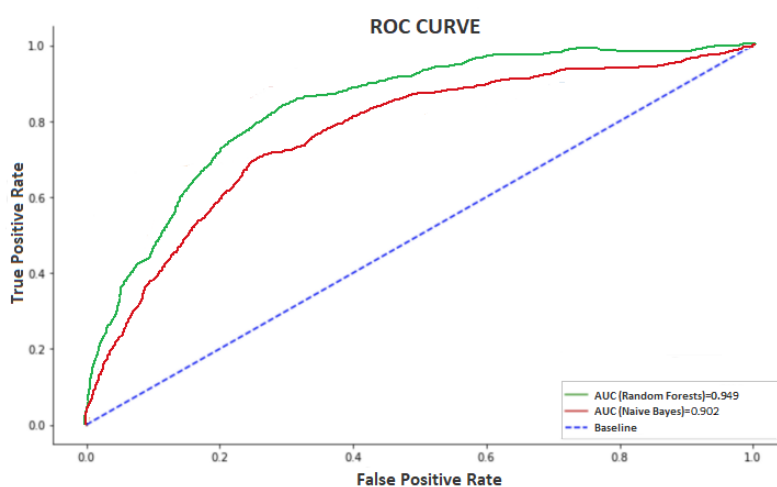


Figura 3.5: Curvas ROC de los modelos evaluados.

Otras Gráficas Relevantes

Sugerencia

Se pueden añadir otras visualizaciones complementarias: evolución del entrenamiento, análisis de errores, importancia de tokens o atención, etc.

3.8. Análisis de Errores

Esta sección tiene como objetivo destallar algunos de los errores cometidos por el sistema, buscando patrones o causas que expliquen los fallos del modelo. No se trata solo de cuantificar los errores, sino de interpretarlos.

Sugerencia

El análisis de errores debe centrarse en identificar los casos en los que el modelo falla sistemáticamente: clases que confunde con frecuencia, tipos de entrada especialmente problemáticos (por longitud, ambigüedad, ruido, etc.), o patrones en los errores.

Errores más frecuentes

Una revisión manual de los errores cometidos por los modelos reveló que gran parte de las instancias mal clasificadas pertenecen a la clase “nombre_clase1”, que tiende a confundirse con “nombre_clase2”. Esto sugiere que la frontera entre estas dos clases no está bien definida en algunos casos.

Identificación de Errores Comunes

Sugerencia

Describir los tipos de errores más comunes que cometió el modelo y las posibles razones. Proporcionar ejemplos específicos de entradas mal clasificadas y discutir posibles mejoras o ajustes al modelo o al preprocesamiento.

Ejemplo 1:

Texto: “A las mujeres hay que respetarlas porque son lo más bello del mundo”

Etiqueta real: Sexista Predicción: No sexista

Comentario: Este mensaje fue clasificado erróneamente como no sexista, probablemente debido a que el clasificador interpreta el lenguaje positivo como respetuoso, sin captar el sesgo implícito de cosificación. Este tipo de error podría corregirse entrenando con ejemplos que incluyan sexismo sutil o benevolente.

Ejemplo 2:

Texto: “Haz lo que quieras, tú siempre sabes más que todos”

Etiqueta real: Sarcástico Predicción: Literal / Acuerdo

Comentario: El modelo no detectó el tono irónico y clasificó el mensaje como afirmativo. Es probable que el sarcasmo indirecto no esté bien representado en los datos de entrenamiento.

Errores por longitud o tipo de texto

Sugerencia

Analiza si hay correlación entre la longitud del texto y el rendimiento del modelo, o si hay ciertos formatos (mensajes muy cortos, uso de emojis, sarcasmo, etc.) que causan más errores.

Al analizar los errores en función de la longitud de los textos, se observó que los mensajes de menos de 10 palabras presentaban una tasa de error significativamente mayor. Muchos de ellos eran frases irónicas o ambiguas.

Reflexiones y posibles mejoras

Sugerencia

Cierra esta sección proponiendo mejoras basadas en el análisis de errores.

Una posible mejora utilizar modelos previamente entrenados sobre detección de ironía o sexismo implícito. También se podrían generar más ejemplos sintéticos, etc..

3.9. Resultados Oficiales de la Tarea (opcional)

Esta sección es opcional y está pensada para aquellos trabajos que hayan participado en una competición oficial. El objetivo es mostrar los resultados enviados durante la fase final de la tarea y contextualizarlos dentro del ranking global.

Sugerencia

Indicar cuáles fueron las estrategias que mejores resultados ofrecieron durante la fase de desarrollo y que se seleccionaron para ser enviadas como *runs* oficiales a la competición.

Durante la fase de desarrollo, se probaron varias configuraciones. La mejor combinación fue la basada en el modelo RoBERTa con aumento de datos mediante backtranslation. Esta estrategia fue seleccionada como uno de los runs oficiales enviados a la competición.

Sugerencia

Describir las características del conjunto de test proporcionado por la competición, si es distinto al de validación. Incluir los resultados oficiales publicados por la organización. Comentar si los resultados fueron similares o no a los esperados.

El conjunto de test oficial incluía 800 ejemplos adicionales, no vistos durante la fase de desarrollo. Los resultados oficiales se muestran en la [Tabla 3.3](#). Aunque nuestro sistema mantuvo un rendimiento estable, el nivel general de los participantes fue más bajo de lo esperado, lo que pone en valor el resultado alcanzado.

Participante	F1-score	Ranking
Equipo A	0.83	1
Equipo B	0.81	2
I2C	0.79	3
Media	0.65	–

Tabla 3.3: Resultados oficiales publicados por la organización de la tarea.

El sistema presentado quedó en tercera posición, por encima de la media global de todos los participantes, lo que demuestra la solidez de la metodología seguida durante el desarrollo del trabajo.

Publicación de los Resultados

Sugerencia

Si se ha contribuido con un artículo científico a las actas del workshop (por ejemplo, IberLEF o CLEF), se debe mencionar aquí y referenciar correctamente. Esto aporta valor académico al trabajo.

Los resultados obtenidos se recogieron en el artículo científico titulado “Título del artículo”, presentado en el workshop IberLEF 2024, y publicado en las actas del evento.

El artículo puede consultarse en [CEUR-Ws²](#). y se ha incluido en un Anexo de esta memoria.

²<http://ceur-ws.org/Vol-XXX>

Conclusiones y Trabajo Futuro

Este capítulo debe ofrecer una síntesis clara del trabajo realizado, valorando los resultados obtenidos y proponiendo posibles trabajos para continuar la investigación. También debe justificarse aquí el esfuerzo temporal invertido.

4.1. Conclusiones

Sugerencia

Describir cómo se han alcanzado los objetivos planteados, cuáles han sido los principales hallazgos, sus implicaciones, posibles limitaciones y la contribución del trabajo dentro del área de estudio.

Consecución de Objetivos

Los objetivos definidos al inicio del trabajo han sido alcanzados satisfactoriamente. Se ha desarrollado un sistema de clasificación basado en Transformers y se ha evaluado en el marco de una competición oficial con buenos resultados.

Resumen de los Hallazgos

El modelo RoBERTa, combinado con técnicas de aumento de datos, ha logrado una mejora del 25 % en F1-score respecto al baseline. El análisis de errores ha revelado que los errores más comunes están relacionados con el lenguaje irónico y ambiguo.

Implicaciones

Los resultados obtenidos demuestran la viabilidad de usar modelos de lenguaje preentrenados para tareas de clasificación de texto en contextos sensibles como la detección de discursos sesgados o polarizados.

Limitaciones

Una limitación importante ha sido el desequilibrio de clases y la falta de ejemplos anotados con ironía sutil o sexismo benevolente, lo que ha dificultado el entrenamiento del modelo.

Contribuciones

Este trabajo ha contribuido con una metodología reproducible para el tratamiento de datos textuales en español, ha evaluado distintas arquitecturas y ha participado activamente en una tarea competitiva, generando un artículo científico.

Consideraciones Éticas

Es importante tener en cuenta las posibles implicaciones éticas del uso de modelos automáticos para clasificar lenguaje humano, especialmente en contextos como el discurso sexista. Las decisiones automatizadas deben siempre contextualizarse y acompañarse de revisión humana.

4.2. Trabajo Futuro

Sugerencia

Proponer líneas de trabajo que den continuidad al proyecto, ya sea en forma de mejoras técnicas, nuevas preguntas de investigación, aplicaciones prácticas o ampliaciones del estudio.

Extensiones del Estudio

Una posible extensión sería aplicar el mismo enfoque a otros dominios del lenguaje subjetivo, como la detección de odio o polarización en redes sociales.

Investigaciones Adicionales

Podría investigarse la eficacia de modelos multilingües o adaptativos en tareas con clases ambiguas o poco representadas.

Aplicaciones Prácticas

Los resultados podrían utilizarse para desarrollar herramientas de apoyo a la moderación de contenido o de análisis sociolingüístico en medios digitales.

Mejoras Metodológicas

Se propone explorar técnicas de aprendizaje activo para mejorar la eficiencia de la anotación de datos y reducir el sesgo en los conjuntos de entrenamiento.

4.3. Planificación Temporal del Trabajo Realizado

En este apartado se muestra el tiempo empleado en el estudio y resolución de las diferentes partes de nuestro TFG/TFM, detallando así las tareas más complejas y las horas globales empleadas en ellas en la [Tabla 4.1](#).

Tabla 4.1: Planificación temporal del trabajo realizado.

Tarea	Horas
Estudio de la Tarea <i>Semeval 2024</i>	5
Aprendizaje de los conceptos necesarios para abordar el TFG:	70
- Aprendizaje automático, redes neuronales, Transformers	
- Transfer Learning, Fine Tuning, PLN, métricas de evaluación	
Aprendizaje de la tecnología necesaria:	80
- PyTorch, modelos de Transfer Learning para NLP	
- Librerías como NLTK, modelos multimodales, L ^A T _E X	
Realización de la tarea de <i>Semeval 2024</i>:	100
- Descarga y estudio de datasets, tratamiento de datos	
- Diseño de experimentos, implementación, entrenamiento	
Elaboración del <i>paper</i> y correcciones	50
Elaboración de la memoria del proyecto	35
Total	340

En esta tabla se refleja el número de horas que se ha dedicado a cada tarea, siendo el desarrollo de la tarea *Semeval 2024* la que más tiempo nos ha llevado, con una diferencia de 20 horas respecto a la siguiente más costosa.

Se han invertido también unas 70 horas en la etapa de formación, estudio de la teoría y los conceptos necesarios para abordar eficazmente la tarea y alcanzar los objetivos del trabajo. La mayor parte de estos conceptos no se abordan en el plan de estudios habitual, lo que nos llevó a buscar material adicional en fuentes especializadas y recursos online.

Se han empleado unas 80 horas en el aprendizaje de las tecnologías necesarias para desarrollar el proyecto. El desarrollo del software necesario para resolver el problema planteado en la competición ha supuesto unas 100 horas.

Una vez completado el plazo de participación, se elaboró el artículo científico y se revisó, lo que supuso unas 50 horas adicionales. Finalmente, para la elaboración y revisión de la memoria del proyecto se dedicaron unas 35 horas.

Referencias

- [1] Laura Plaza et al. “Overview of exist 2025: Learning with disagreement for sexism identification and characterization in tweets, memes, and tiktok videos”. En: *International Conference of the Cross-Language Evaluation Forum for European Languages*. Springer. 2025, págs. 266-289.
- [2] Lin Tian, Johanne R Trippas y Marian-Andrei Rizoio. “Mario at EXIST 2025: a simple gateway to effective multilingual sexism detection”. En: *arXiv preprint arXiv:2507.10996* (2025).
- [3] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [4] Christopher M Bishop. *Pattern recognition and machine learning*. 2006.
- [5] Trevor Hastie, Robert Tibshirani, Jerome Friedman et al. *The elements of statistical learning*. 2009.
- [6] Andrew G Barto. “Reinforcement learning: Connections, surprises, challenges”. En: *AI Magazine* 40.1 (2019), págs. 3-15.
- [7] Min-Ling Zhang y Zhi-Hua Zhou. “A review on multi-label learning algorithms”. En: *IEEE transactions on knowledge and data engineering* 26.8 (2013), págs. 1819-1837.
- [8] Shervin Minaee et al. “Deep learning-based text classification: a comprehensive review”. En: *ACM computing surveys (CSUR)* 54.3 (2021), págs. 1-40.
- [9] Andrej Karpathy et al. “Large-scale video classification with convolutional neural networks”. En: *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 2014, págs. 1725-1732.
- [10] Tadas Baltrušaitis, Chaitanya Ahuja y Louis-Philippe Morency. “Multimodal machine learning: A survey and taxonomy”. En: *IEEE transactions on pattern analysis and machine intelligence* 41.2 (2018), págs. 423-443.
- [11] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <https://www.deeplearningbook.org>.
- [12] Yann LeCun, Yoshua Bengio y Geoffrey Hinton. “Deep learning”. En: *Nature* 521.7553 (2015), págs. 436-444.
- [13] Francois Chollet y Matthew Watson. *Deep learning with Python*. Vol. 2. Manning publications Shelter Island, NY, USA, 2021.
- [14] Lutz Prechelt. “Early stopping-but when?” En: *Neural Networks: Tricks of the trade*. Springer, 2002, págs. 55-69.
- [15] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. En: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, págs. 437-478.
- [16] Dominic Masters y Carlo Luschi. “Revisiting small batch training for deep neural networks”. En: *arXiv preprint arXiv:1804.07612* (2018).
- [17] Sebastian Ruder. “An overview of gradient descent optimization algorithms”. En: *arXiv preprint arXiv:1609.04747* (2016).

- [18] Anders Krogh y John Hertz. “A simple weight decay can improve generalization”. En: *Advances in neural information processing systems* 4 (1991).
- [19] Ilya Loshchilov y Frank Hutter. “Decoupled weight decay regularization”. En: *arXiv preprint arXiv:1711.05101* (2017).
- [20] Shai Shalev-Shwartz y Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.
- [21] Kevin P Murphy. *Machine learning: a probabilistic perspective*. MIT press, 2012.
- [22] Jinseok Nam et al. “Large-scale multi-label text classification—revisiting neural networks”. En: *Joint european conference on machine learning and knowledge discovery in databases*. Springer. 2014, págs. 437-452.
- [23] Xue Ying. “An overview of overfitting and its solutions”. En: *Journal of physics: Conference series*. Vol. 1168. 2. IOP Publishing. 2019, pág. 022022.
- [24] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. En: *The journal of machine learning research* 15.1 (2014), págs. 1929-1958.
- [25] Mohammad Mahdi Bejani y Mehdi Ghatee. “A systematic review on overfitting control in shallow and deep neural networks: MM Bejani, M. Ghatee”. En: *Artificial Intelligence Review* 54.8 (2021), págs. 6391-6438.
- [26] James Bergstra y Yoshua Bengio. “Random search for hyper-parameter optimization.” En: *Journal of machine learning research* 13.2 (2012).
- [27] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. En: *arXiv preprint arXiv:1810.04805* (2018).
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar et al. “Attention is all you need”. En: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [29] Sebastian Ruder. “Neural Transfer Learning for Natural Language Processing”. En: *arXiv preprint arXiv:1903.11260* (2019). URL: <https://arxiv.org/abs/1903.11260>.

A.1. Aplicación Web para la Visualización y Prueba del Modelo

Como parte complementaria del trabajo, se ha desarrollado una aplicación web que permite cargar el modelo entrenado y utilizarlo para clasificar nuevas instancias de texto.

Esta aplicación proporciona una interfaz sencilla e intuitiva para que cualquier usuario pueda interactuar con el modelo sin necesidad de conocimientos técnicos.

Tecnologías Utilizadas

- **Backend:** Python con `FastAPI`, modelo cargado con `transformers`.
- **Frontend:** HTML, CSS y JavaScript con `Bootstrap`.
- **Despliegue:** Aplicación alojada en *HuggingFace Spaces*.

Acceso a la Aplicación

La aplicación puede accederse desde la siguiente URL:

<https://miPagina.es>

Manual de Usuario

1. Acceder a la URL anterior desde cualquier navegador.
2. Introducir el texto que se desea clasificar en el campo correspondiente.
3. Pulsar el botón “Clasificar”.
4. Se mostrará la predicción del modelo y, opcionalmente, las probabilidades asociadas a cada clase.

Ejemplo de uso

Texto introducido: “No me parece bien lo que estás diciendo.” *Salida del modelo:* Clase: Desacuerdo

Capturas de pantalla